x86-64 Assembly Language

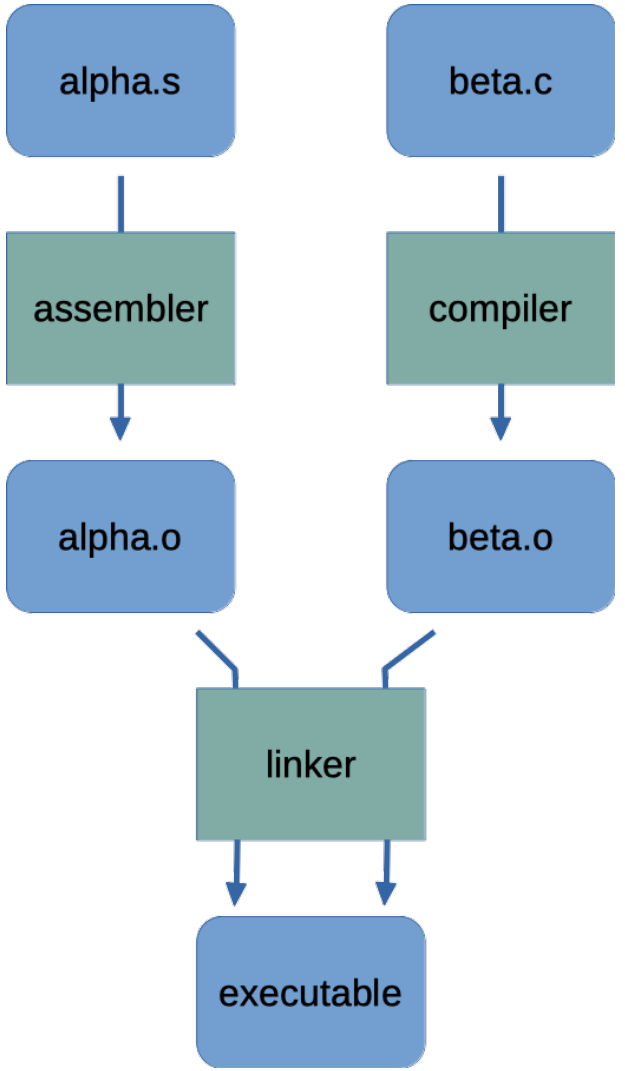
CSC 230

Assembly Language

- Programming for the Intel & AMD 64-bit instruction set
  - Called x86-64
  - This is what gcc compiles to on the common platform
  - Used on most modern desktop and laptop systems
  - … although Apple has started using ARM.
  - … phones and tablets typically use ARM also.
- There's a long history of this instruction set
  - Going back to the 8086 processor (1978)
  - The 80286 (1982) … the 80386 (1985) … the i486 (1989) and lots of others.
  - So it's often called the x86
  - x86-64 is the modern, 64-bit architecture and instruction set.

Building an executable

- Source files are in *assembly language*.
  - Some people call this assembler language, or just assembler.
- Using human-readable *mnemonics* (e.g., add, mov, ret) for machine instructions.
- Source files end with a **.s**
- We can run them through an *assembler* to get an object file
  - Just like how we run a C program through a compiler.
  - We can link this with other objects to get an executable.



Hello World

```
    ## Program to print out Hello World.

    .global main

    .text

message:
    ## String to print out.
    .asciz  "Hello World."

main:
    ## Call to puts( "Hello World." );
    mov     $message, %rdi          # Address of message is the parameter.
    call    puts                    # Call puts()
    ret                             # Return from main.
```

Building and Running Hello World

- The gcc compiler knows how to assemble and link:

```
$ gcc hello.s -o hello
```

- The command above should work on the common platform.
- On a newer Linux installation, you may need disable position-independent executable

```
$ gcc -no-pie hello.s -o hello
```

- Then, you can run it like any other program:

```
$ ./hello
```

Hello World Explained

Comments
They start with a pound sign.

```
### Program to print out Hello World.

    .global main

    .text

message:
    ## String to print out.
    .asciz  "Hello World."

main:
    ## Call to puts( "Hello World." );
    mov     $message, %rdi          # Address of message is the parameter.
    call    puts                    # Call puts()
    ret                             # Return from main.
```

Hello World Explained

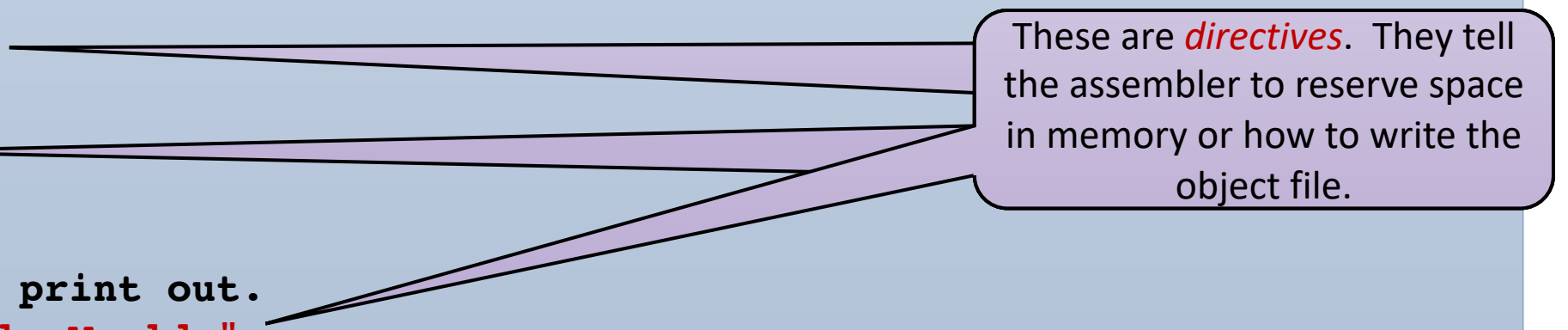
```
### Program to print out Hello World.

    .global main

    .text

message:
    ## String to print out.
    .asciz  "Hello World."

main:
    ## Call to puts( "Hello World." );
    mov     $message, %rdi          # Address of message is the parameter.
    call    puts                    # Call puts()
    ret                             # Return from main.
```



Hello World Explained

```
### Program to print out Hello World.

    .global main

    .text

message:
    ## String to print out.
    .asciz  "Hello World."

main:
    ## Call to puts( "Hello World." );
    mov     $message, %rdi          # Address of message is the parameter.
    call    puts                    # Call puts()
    ret                             # Return from main.
```

These are labels. They let us talk about a location in our code or where something is stored.

Hello World Explained

```
### Program to print out Hello World.

    .global main

    .text

message:
    ## String to print out.
    .asciz  "Hello World."

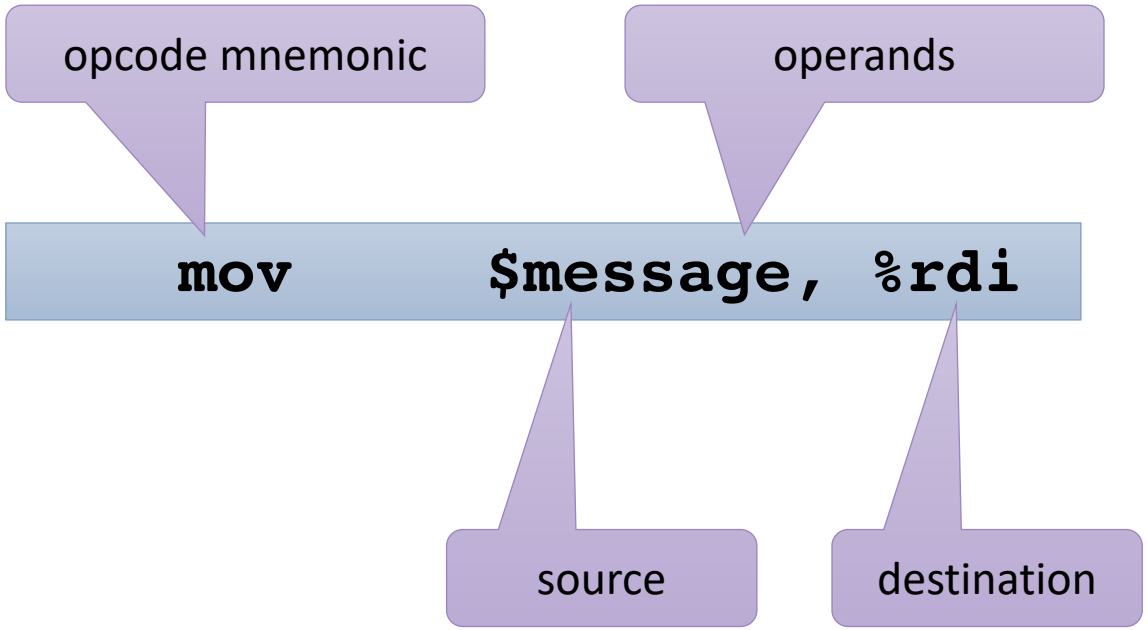
main:
    ## Call to puts( "Hello World." );
    mov     $message, %rdi          # Address of message is the parameter.
    call    puts                    # Call puts()
    ret                             # Return from main.
```

These are instructions. They say what the program will do when it is run.

Still using the C standard library here.

Assembly Instructions

- A typical instruction looks like this:



- It has an *opcode mnemonic*
  - This says what the instruction does
- It may have *operands*
  - This says what it does it to
  - Instructions may have zero, one or two operands
- The last operand is the *destination*
- The operand before that is the *source*.

Syntax Variation

- We're using AT&T syntax for assembly language.
  - That's what the gcc compiler suite uses by default.
- Intel syntax is also common
  - Of course, it uses the same instructions
  - But, it notates instructions, operands and directives differently
  - We won't use this (but it's good to know it's out there).
  - CSC 236 currently uses Intel syntax

```
mov     $message, %rdi
```

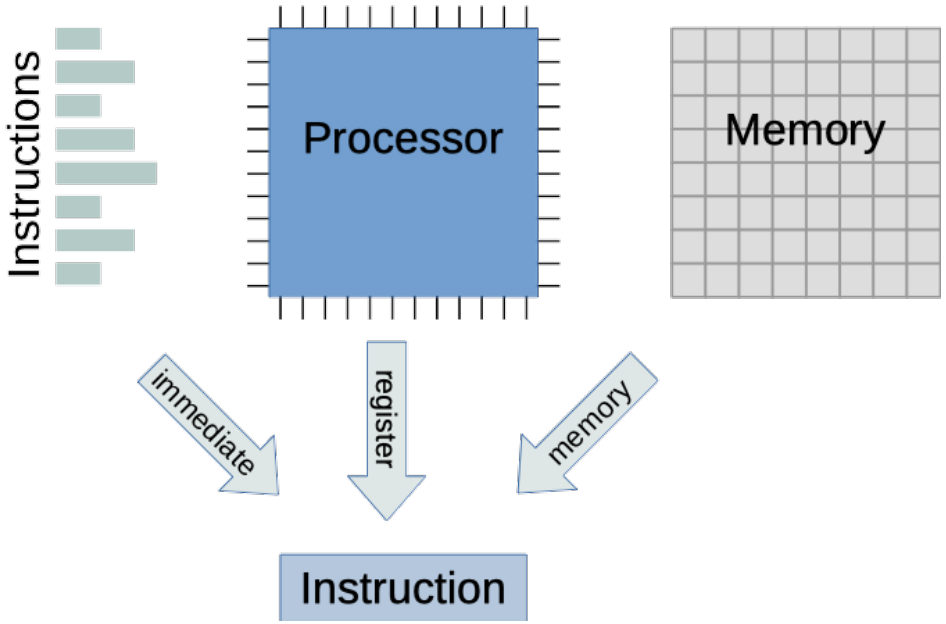
AT&T Syntax

```
mov     rdi, offset message
```

Intel Syntax

Operand Locations

- From memory
  - There are several *addressing modes* to let you choose the location
  - Can't have more than one operand from memory
- Part of the instruction
  - This is called an *immediate* operand
  - They have a dollar sign ($) in front of the value.
- The processor
  - The CPU has a small amount of space for holding values you're working with right now.
  - These are the *registers*
  - They have a percent (%) in front of the name.



Data Sizes

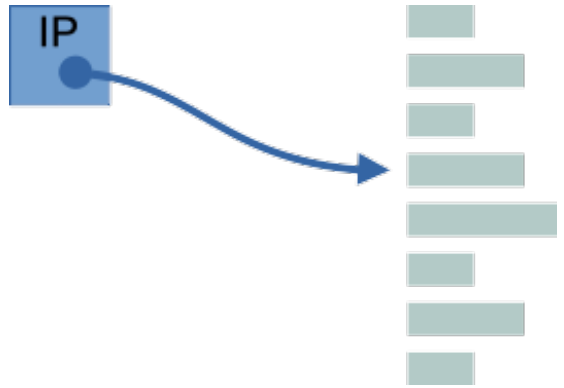
- X86-64 can work with data of different sizes.
- The following integer sizes:

x86-64 Name	Size	Common Platform C
byte	8 bits	char
word	16 bits (2 bytes)	short
longword	32 bits (4 bytes)	int
quadword	64 bits (8 bytes)	long, long long, pointer

- It supports other data types
  - For things like floating point operations
  - … but we won't be using those.

x86-64 Registers

- There are 16 registers available to the programmer
  - Plus, some special-purpose ones like the *instruction pointer* : keeps up with the address of the next machine instruction
  - Or the *status register* : keeps up with the result of previous computations or tests
- Naming of the registers is a little bit irregular
  - Reflecting the history of the processor
    4 registers, 8-bits/16-bits each →
    8 registers, 16-bits each →
    8 registers, 32-bits each →
    16 registers, 64 bits each



Registers as 64 bits or 32 bits

- **rax** is the name for the first, 64-bit register.
  - You can think of this as “register ax”

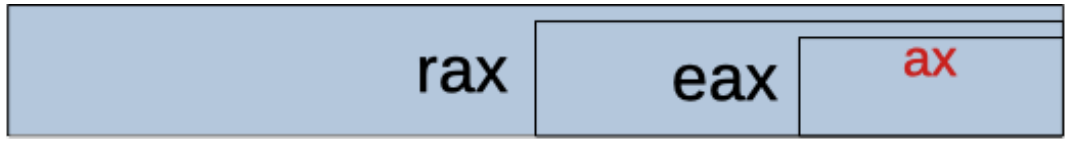


- If you call this register **eax**, you get just the least-significant 32 bits.
  - You can think of this as “extended ax” (for historic reasons)



Registers as 16 bits or 8 bits

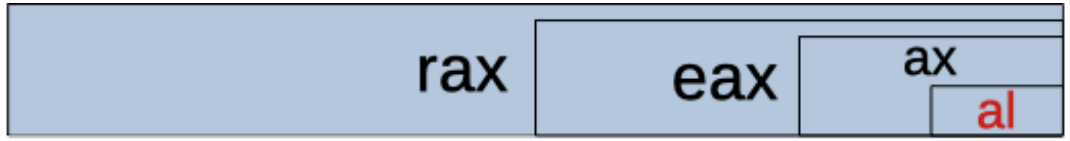
- If you call it just **ax**, you get the least-significant 16 bits.
  - This is where the name ax came from. It's both halves of the **a** register.



07B2304C7EA629D4

That's the low-order 16 bits.

- If you say al (lower-case AL), you get the least-significant 8 bits.

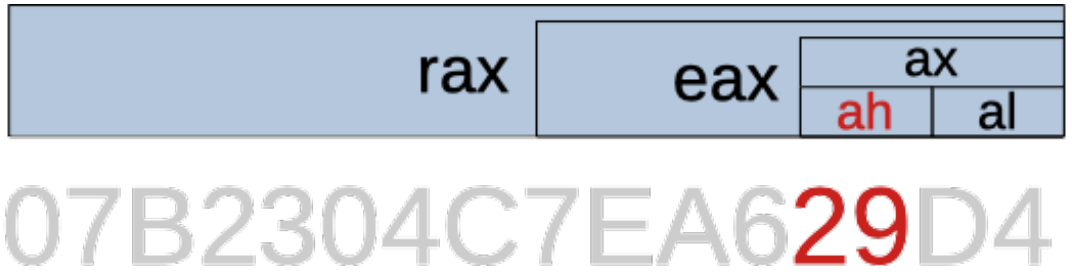


07B2304C7EA629D4

Two digits of hexadecimal. That's 8 bits.

Registers as 8 higher-order bits

- For some of the older registers, you can get bits 8 - 15
- The name **ah** (high part of ax) will give you these bits.

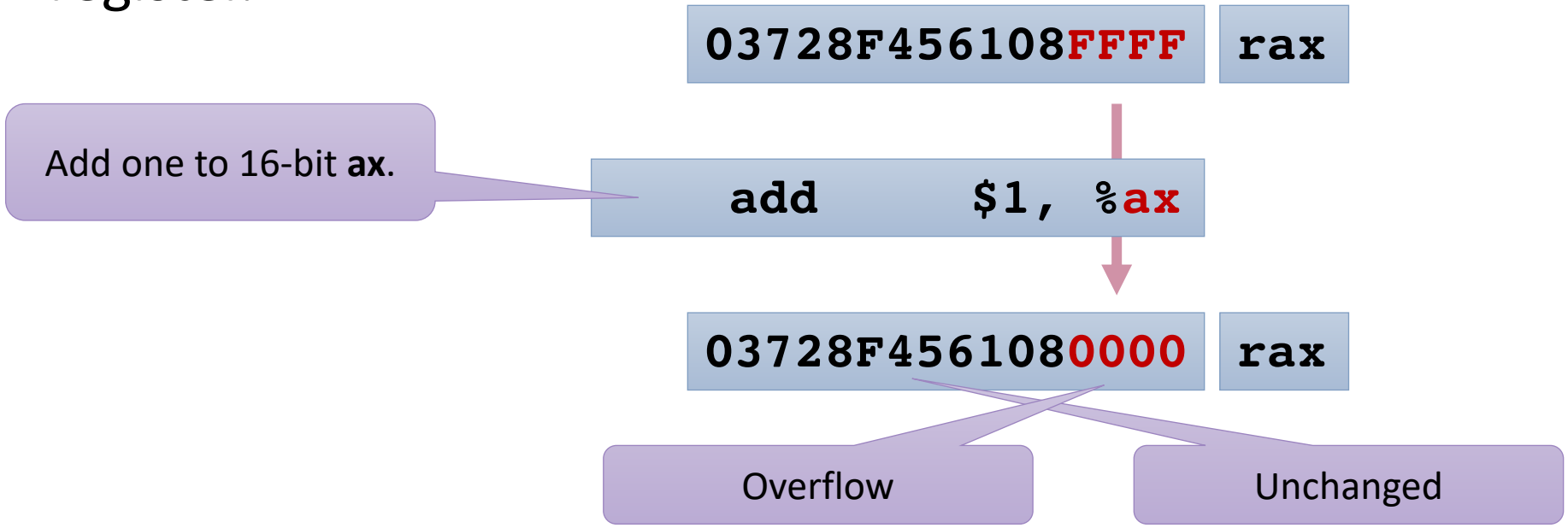


That's the 8 bits from the second-least-significant byte.

- Not all registers let you get these particular bits.
- Ideally, you wouldn't need to know the history of an instruction set to understand it.
  - … but it does explain some things and make them easier to remember.

Register Isolation

- These names us you talk about different parts of a register.
- This can give us a way to convert data types.
- When we operate on one part, it doesn't overflow into the rest of the register.

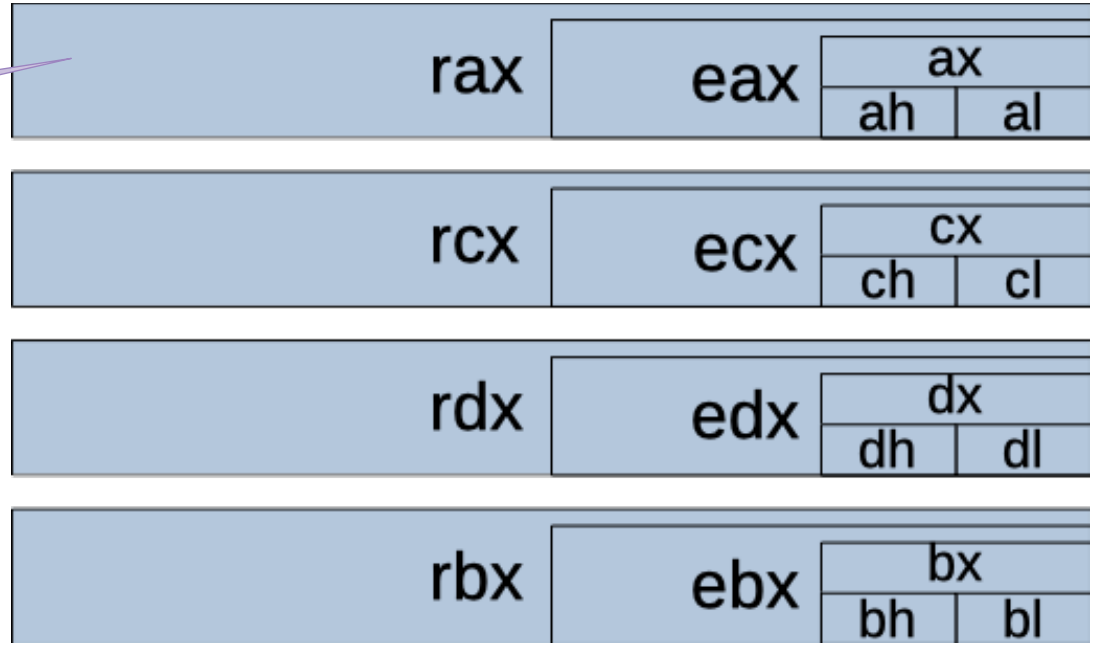


The Rest of the Registers

- 8086 had 4 general-purpose registers, named **ax**, **cx**, **dx** and **bx**.
- These have been extended to **rax**, **rcx**, **rdx** and **rbx**.
- You can access the parts of these registers just like you can with **rax**.

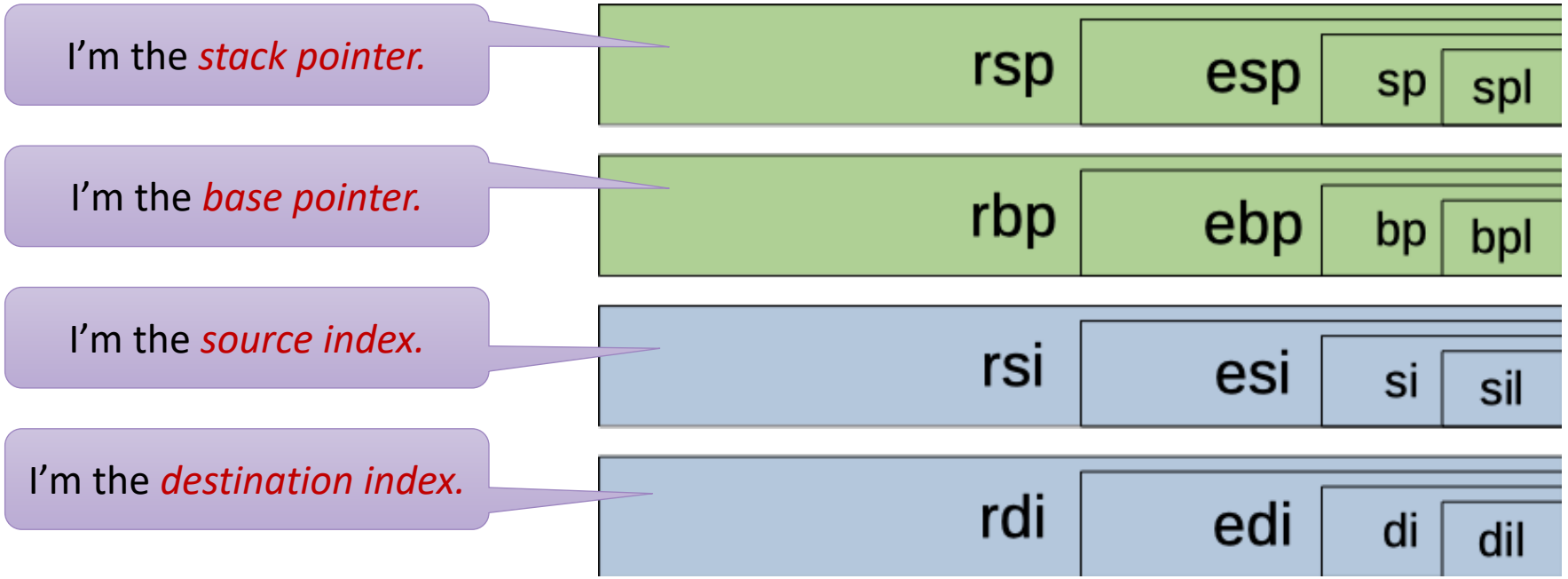
Notice the weird order

I'm sometimes called the accumulator.



More, Old Registers

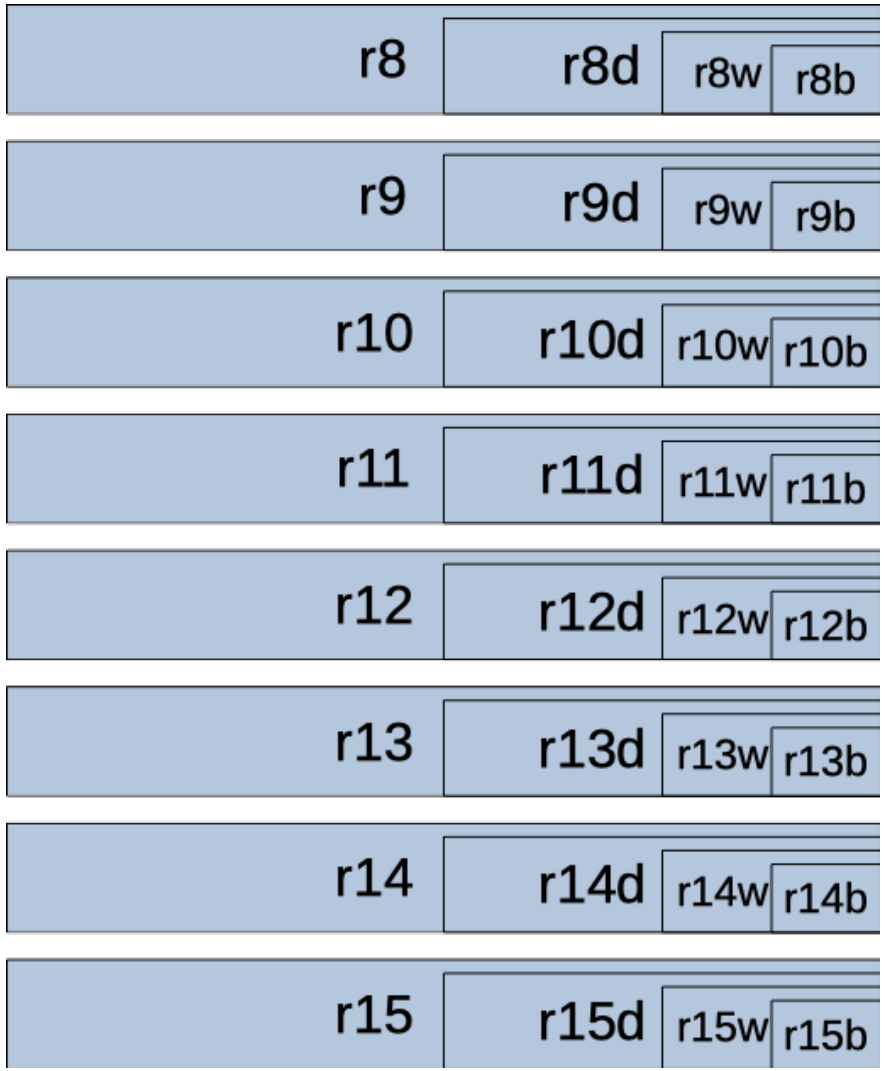
- The 8086 had 4 more registers used for particular jobs.
  - We can't get the second-to-last byte with these.
  - The first two are generally for working with the stack.



Meet the New Registers

- The 64-bit instruction set gave us 8 more registers.
  - Naming for these got less creative.
  - This is more like the naming scheme in a RISC processor (e.g., ARM)
- Notice, there's a different naming scheme for getting the low-order bits
  - Longword (doubleword, d)
  - Word (w)
  - Byte (b)

Notice the different naming scheme.



Registers in C

- You can't use registers directly in C ... but you can give suggestions.
- Local variables are normally stored on the stack.
- We can make a recommendation that the compiler use a CPU register instead.

```
int total = 0;
for ( int i = 0; i < 1000000000; i++ )
  total += i;
```

Allocated on the stack.

```
int total = 0;
for ( register int i = 0; i < 1000000000; i++ )
  total += i;
```

Consider storing i in a CPU register.

Making Suggestions

- `register` can only be specified for **`auto`** variables (i.e., not for global variables)
  - There are other restrictions on what types of variables can be specified as **`register`**
  - And, if you ask for a variable to be register, you can't take its address!
- Less important for a modern compiler
  - Optimizing compilers may be able to do a better job than you at allocating scarce CPU registers

Your First Instruction : mov

- mov *source*, *dest*
  - Copy a value from *source* to *dest*.
- Size of the data may be determined by context

```
mov     %ax, %r8w
```

You must want a word-sized move.

- Or, there are single-character suffixes to specify the data size.

```
movb    $1, variable
```

OK. You want me to move a byte.

- The compiler will complain if it doesn't know how much you want to move.

```
… Error: no instruction mnemonic suffix given …
```

Mnemonic Suffixes

- There are mnemonic suffixes for each of the data sizes
- For byte : b

```
movb      $1, variable
```

- For word : w

```
movw      $1, variable
```

- For longword : l

```
movl      $1, variable
```

- For quadword : q

```
movq      $1, variable
```

Starter Operand Types

- A few basic operand types will get us pretty far
- Immediate : Just a number, up to 32 bits
  - The instruction can contain a value as the operand
  - A dollar sign ($) in front of the value:

```
mov     $25, %rax
```

rax will get 0x0000000000000019

  - You can give the value as a signed, decimal value:

```
mov     $-1, %rcx
```

rcx will get 0xFFFFFFFFFFFFFFFF

  - You can give the value in hexadecimal:

```
mov     $0xEF17, %r9d
mov     $'c', %bl
```

r9 will have 0x????????0000EF17

rbx will get 0x????????????????63

Starter Operand Types

- Register : Contents of a Register
  - Name of a register (or part of one) with a % in front.

```
mov     $25, %rax
```

  - You can use a register as a source or a destination.

```
mov     %ax, variable
```

- Displacement : Access a particular address in memory
  - Give the name of a global variable
  - For accessing its contents, **you** have to know how big it is.

```
movb     $1, variable
```

```
mov      variable, %eax
```

More Instructions : Basic Math

- There are instructions for typical arithmetic and logical operations.
- **add *source, dest***
  - *dest* + *source* → *dest*
  - Unsigned or signed addition

```
add     $5, %eax
```

- **sub *source, dest***
  - *dest* - *src* → *dest*
  - Unsigned or signed addition

```
sub     %ax, %cx
```

- These accept any of the mnemonic suffixes for size.

More Instructions : Basic Math

- There are simplified functions to add / subtract 1.
- **inc *dest***
  - *dest* + 1 → *dest*
  - Unsigned or signed addition

```
inc      %cl
```

- **dec *dest***
  - *dest* – 1 → *dest*
  - Unsigned or signed addition

```
dec      %r8
```

- These accept any of the mnemonic suffixes for size.

More Instructions : Basic Math

- Multiply and divide are more complicated.
- They depend on the signed-ness and size of the operands
- There's a simple, signed, two-parameter multiply
- **imul *source, dest***
  - *dest* * *source* → *dest*
  - Destination must be a register

```
imul      %ecx, %r9d
```

  - Works for word, longword or quadword (not byte)
- There are other multiply operations for unsigned, byte, etc.

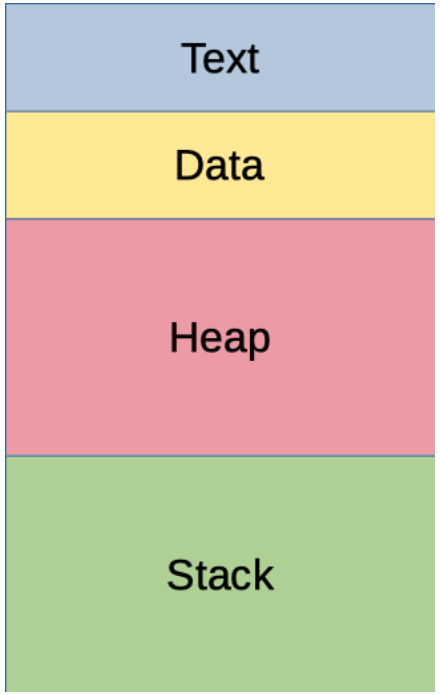
Debugging Assembly

- You can use gdb to trace through assembly code.
- This is good
  - We will write code that combines C and assembly.
  - And we already know how to use some of gdb.

Command	Abbrev	
break *label*	b *label*	set a breakpoint at the given label
break *address*	b *address*	set a breakpoint at the given address
print */fmt expr*	p */fmt expr*	print the value of the expression in the given format d = decimal, x = hexadecimal, b = binary, i = instruction (e.g., p /x var, p /i main)
info register x	i r *x*	print the value of register x (e.g., i r rsi, i r al)
info registers	i r	print all registers (including some we're not talking about)

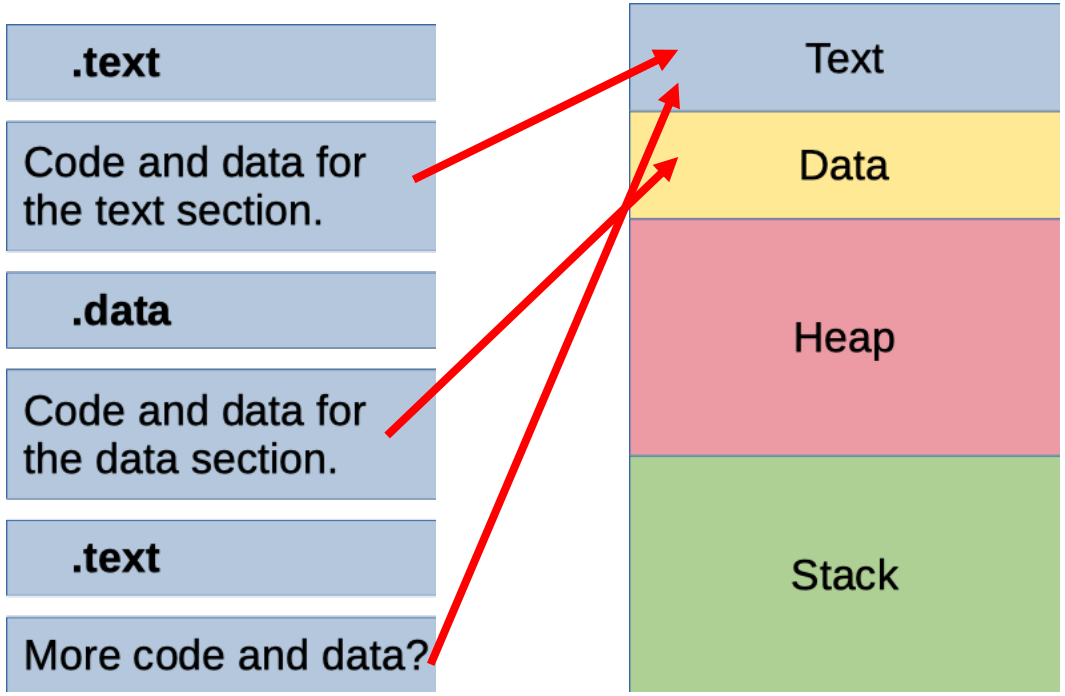
Program Organization

- Programs have four main regions in memory.
- The heap and stack get populated as the program runs
- The text and data sections get created by the compiler and linker, before the program starts running.
  - They’re both static, fixed-sized, fixed-organization.
- The text section is for code and read-only, static data (e.g., literal strings)
- The data section is for modifiable static data (e.g., global variables)



Program Organization

- Assembler directives let you put things in text or data.
  - **.text** : put subsequent code and data in the text section
  - **.data** : put subsequent code and data in the data section.
- You can switch back and forth between these sections.
- Each object file can add some things to these sections.



Data Declarations

- With data declarations, you can
  - Reserve space for globals and other static storage
  - Initialize static data
  - Use labels to give names to particular storage locations.

```
        .data
v1:     .byte   10
v2:     .byte   20, 30, -10
v3:     .byte   'a', 'b', 'c'
v4:     .word   0x28C2
v5:     .asciz  "xyz"
```

You can initialize values in decimal.

You can give multiple consecutive values.

Negative values will use two's complement.

I'm a label, a name for a memory location.

Data Declarations

- With data declarations, you can
  - Reserve space for globals and other static storage
  - Initialize static data
  - Use labels to give names to particular storage locations.

```
        .data
v1:     .byte   10
v2:     .byte   20, 30, -10
v3:     .byte   'a', 'b', 'c'
v4:     .word   0x28C2
v5:     .asciz  "xyz"
```

Single character literals give you the ASCII value

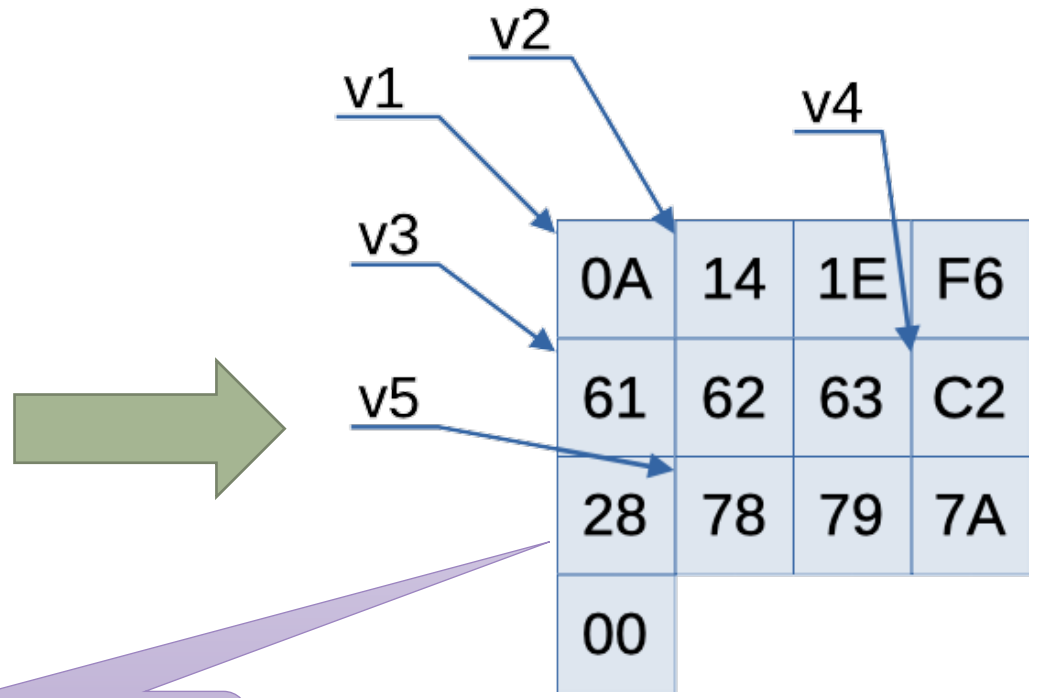
You can give values in hexadecimal.

String values allocate consecutive ASCII values.

Data Declarations

- Values get reserved in the sizes and in the order you ask for.

```
        .data
v1:     .byte   10
v2:     .byte   20, 30, -10
v3:     .byte   'a', 'b', 'c'
v4:     .word   0x28C2
v5:     .asciz  "xyz"
```



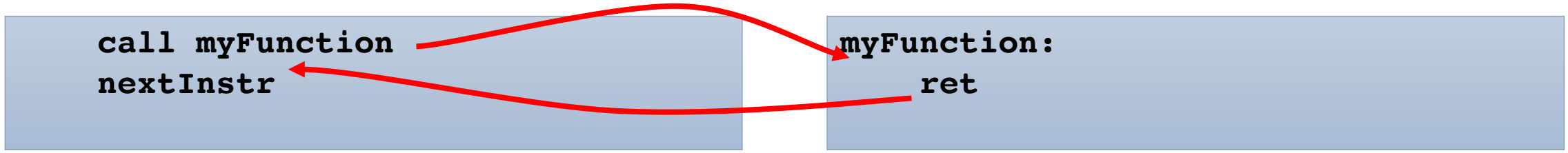
Data Declarations

- There are directives for standard data types.

Directive	Data allocation
.byte	Create one or more 8-bit values.
.word	Create one or more 16-bit values, each in little endian byte order.
.long	Create one or more 32-bit values, each in little endian byte order.
.quad	Create one or more 64-bit values, each in little endian byte order.
.ascii	Create a sequence of characters (without null termination).
.asciz	Create a null-terminated string.

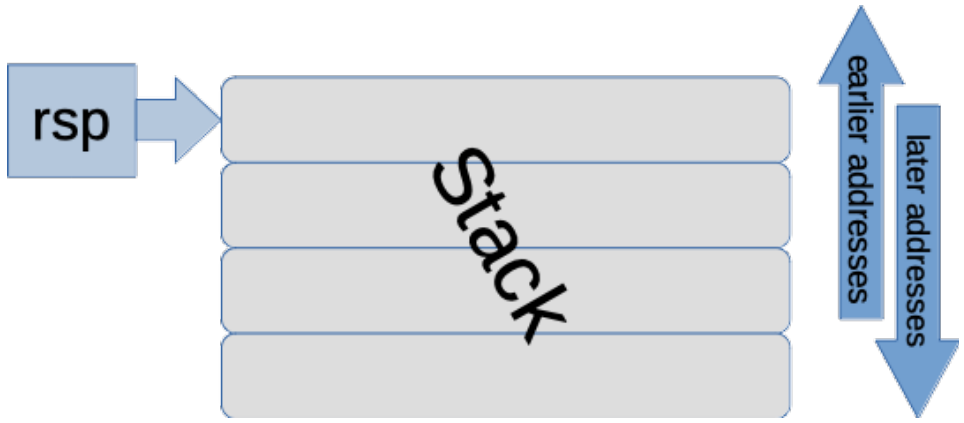
Your First Assembly Language Function

- The simplest function
  - No parameters, no return value.
  - A label for the functions starting instruction.
  - Return immediately from the function
- New instructions
  - **call *label***
    - Save the return return address and jump to the given label.
  - **ret**
    - Return to the next instruction after the call.



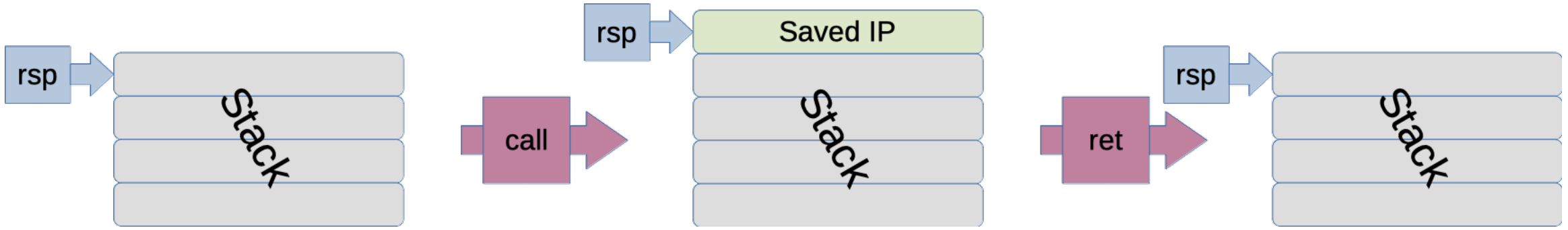
The Stack

- Assembly programs make use of the same stack as C programs.
- Registers **rsp** and **rbp** keep important stack locations.
  - **rsp** points to the top value on the stack
  - **rbp** normally points to the stack frame for the current function.
  - The stack grows toward lower-numbered memory addresses.
- Lots of people draw the stack growing downward
- It just depends on which way you think increasing memory addresses should go.



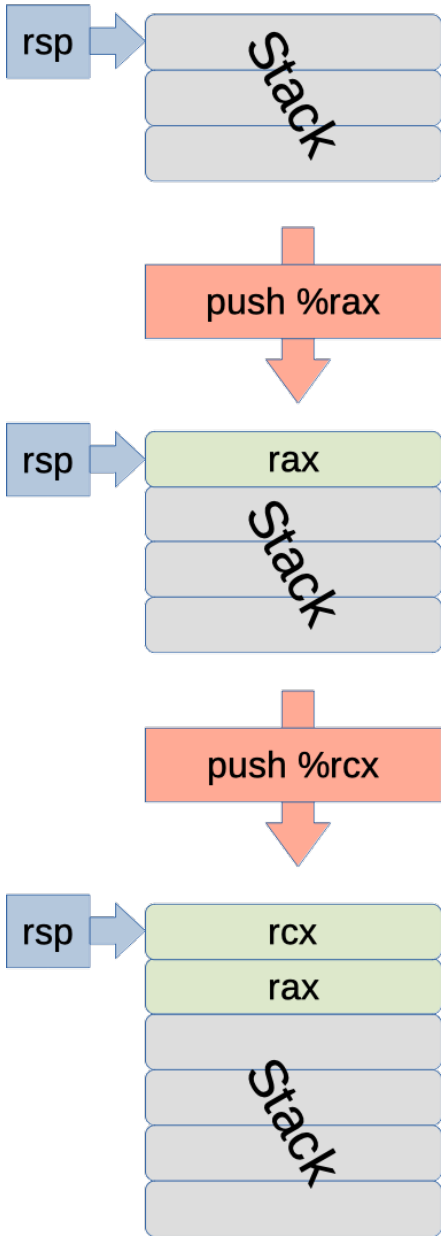
The Stack and Subroutine Calling

- The call instruction pushes the Instruction Pointer onto the stack
- The ret instruction pops it and puts it back in the Instruction Pointer
- So, the stack before returning, the stack must look just like it did when the function started.



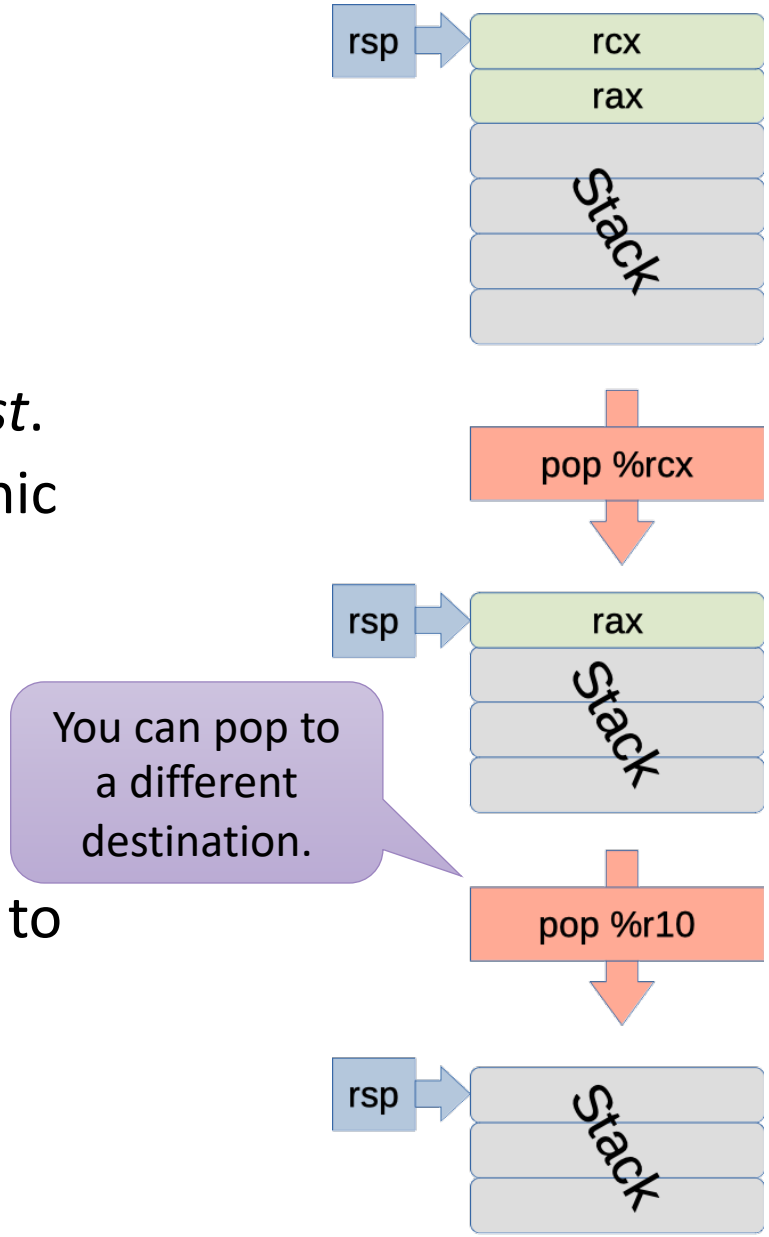
Using the Stack

- The stack is used for storing lots of things.
  - Allocating space for local variables.
  - Saving / restoring copies of important registers.
  - Passing (some) parameters to a function.
- **push *source***
  - Push a copy of *source* onto the stack
  - Must be word-sized or quadword-sized (will normally be a quadword)
  - Decrements the stack pointer by operand size.
  - Leaves **rsp** pointing at the top item on the stack.



Using the Stack

- **pop *dest***
  - Remove a value from the stack and store it in *dest*.
  - Must be word-sized or quadword-sized (mnemonic suffixes for these)
  - Increments the stack pointer by 8 or 2 bytes.
  - Leaves rsp pointing at the top item on the stack.
- Math on the stack pointer.
  - To discard items from the stack, you can just add to the stack pointer.
  - Reserving (uninitialized) space on the stack just requires subtracting from the stack pointer.



Calling Conventions

- *Calling conventions* give the rules for how to call a function (with particular hardware, a particular OS and in a particular language)
- We'll use the call / ret instructions, but there's more than that.
  - How are function parameters passed?
  - How are values returned to the caller?
  - Is the function free to use and modify all registers? (not all of them)
  - Alignment expectations? (yes)
- We also need to make sure the calling language knows about the function it's trying to call.

Calling Assembly Functions from C

- Calling assembly functions from C takes some work.
- We have to tell the assembler to make the function's label visible.
  - This is done with a **.global** assembler directive.
- C will also want a prototype for the function
  - So it knows how to call it properly.

A prototype, to tell C about our function.

```
void doNothing();
```

```
doNothing();
```

Let the linker see the function's label.

```
.global doNothing
```

```
doNothing:
    ret
```

Calling C Functions from Assembly

- The other way is (maybe) easier.
- Assembly language doesn't care about prototypes
  - It's your job to make sure you use any symbol correctly.
- Non-static symbols in C are automatically exposed to the linker.

```
void doNothing()
{
}
```

This symbol is visible to the linker.

```
    ...
    call doNothing
    ...
```

The linker will find this symbol for me.

Returning Values

- The **rax** register is used to return small values
  - Use **eax** (for int, short or char) or **rax** (for long, long long or pointers)

```
int getValue();
```

```
   x = getValue();
```

```
    .global getValue
```

```
getValue:
    mov    $35, %eax
    ret
```

- Larger return values can be returned via the stack
- There are other registers for returning floating-point results.

We won't cover these.

Accessing C Globals from Assembly

- Just like functions, other non-static C symbols are visible to assembly code.
- Assembly code can access them by name.
- No extern declarations is required.

```
int aPlusB()
{
    return globalA + globalB;
}
```

```
int aPlusB();
int globalA = 5;
int globalB = 7;
```

```
    x = aPlusB();
```

```
    .global aPlusB
```

```
aPlusB:
    mov     globalA, %eax
    add     globalB, %eax
    ret
```

Parameter Passing Conventions

- The first six integer values or pointers are passed in CPU registers.
  - After that, we'll have to use the stack.
  - Or, for floating-point parameters, there are other registers.
- Use the 64-bit register (for long or pointer), or just the 32-bit part (for int, short or char)

We won't cover these.

Parameter	Register
First integer / pointer parameter	rdi
Second integer / pointer parameter	rsi
Third integer / pointer parameter	rdx
Fourth integer / pointer parameter	rcx
Fifth integer / pointer parameter	r8
Sixth integer / pointer parameter	r9

Using Parameters

- For integer parameters, you can access your own parameters via these registers

```
int addNumbers( int, int );
```

```
x = addNumbers( 5, 7 );
```

```
    .global addNumbers
```

```
addNumbers:
    mov     %edi, %eax
    add     %esi, %eax
    ret
```

```
int addNumbers( int d, int s )
{
    return d + s;
}
```

We're using these two parameters.

Passing Parameters

- You can call C functions inside your assembly code.
- Passing them parameters using the same calling conventions.

```
myFunction:
    mov     $'a', %edi
    call    putchar
    mov     $'b', %edi
    call    putchar
    mov     $'c', %edi
    call    putchar
    mov     $'\n', %edi
    call    putchar
    ret
```

Call putchar() to print a letter a.

Call it three more times.

```
void myFunction()
{
    putchar( 'a' );
    putchar( 'b' );
    putchar( 'c' );
    putchar( '\n' );
}
```

Passing Parameters

- Pointer parameters and long integers require the whole 64-bit register.
- Here, we're calling malloc, then freeing the block we just allocated.

```
myFunction:
    mov     $100, %rdi
    call    malloc
    mov     %rax, %rdi
    call    free
    ret
```

```
void myFunction()
{
    char *x = malloc( 100 );
    free( x );
}
```

Size requires the whole 64-bit register.

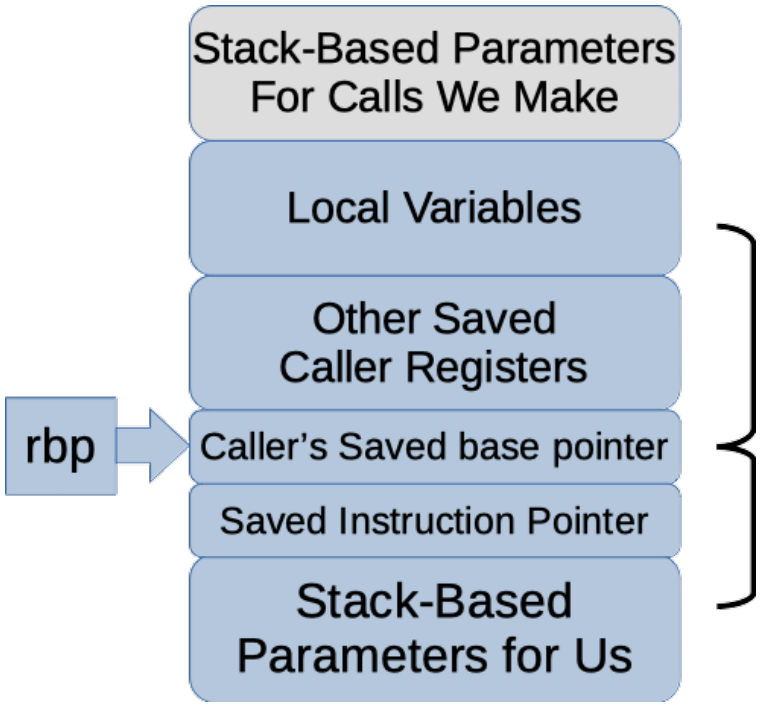
A pointer also requires 64 bits.

Saving Registers

- In general, a function can use any registers it wants.
- This will overwrite whatever the caller had in these registers.
- That's OK … for most registers … but there are a few the subroutine can't use.
  - Or, at least, it must save what's in them and restore it before returning.
  - The stack is the usual place to save these.
- The callee-saved registers are: **rbx**, **rsp**, **rbp**, **r12**, **r13**, **r14** and **r15**

Creating a Stack Frame

- A function will normally use a *stack frame* to
  - Access parameters passed to it on the stack.
  - Save copies of registers the caller may be using
  - Create space for local variables
  - Stage stack-based parameters for functions it will call.
- The base pointer (rbp) was made to be a pointer to this location on the stack.
  - Important values for the function are at a fixed displacement from the base pointer.
- There's a standard way to set it up.



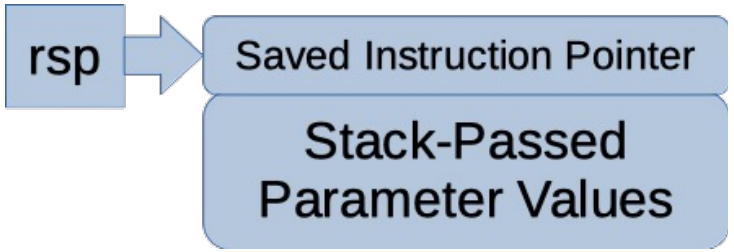
Creating a Stack Frame

- When a function starts running, the stack looks like this.

```
subroutine:

    ...

    ret
```



Creating a Stack Frame

- We'll save a copy of **rbp** on the stack
- It's on the list of callee-saved registers

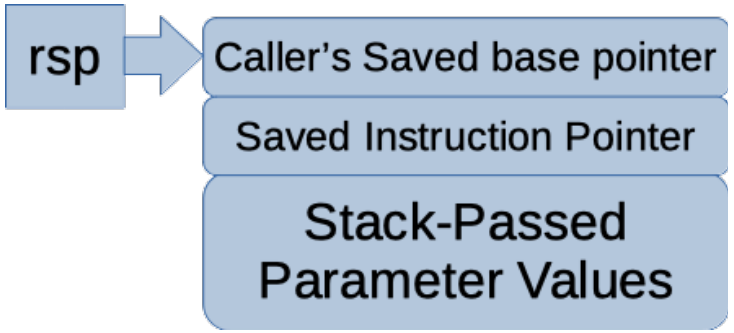
```
subroutine:
    push    %rbp

    ...

    pop     %rbp
    ret
```

Save it at the start

Restore it when we're done.



Creating a Stack Frame

- Then, we'll copy the stack pointer to rbp
- The stack pointer will move (as we push/pop), but rbp will stay here.

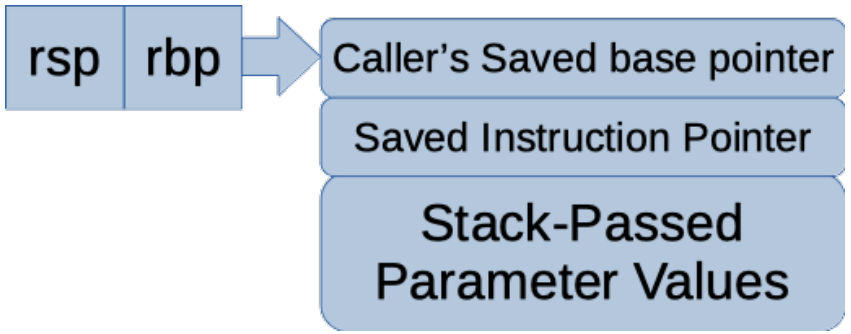
```
subroutine:
    push    %rbp
    mov     %rsp, %rbp




    ...




    pop     %rbp
    ret
```



Creating a Stack Frame

- We can save other caller registers if we need to.
- This moves the stack pointer … but the base pointer stays where it is.

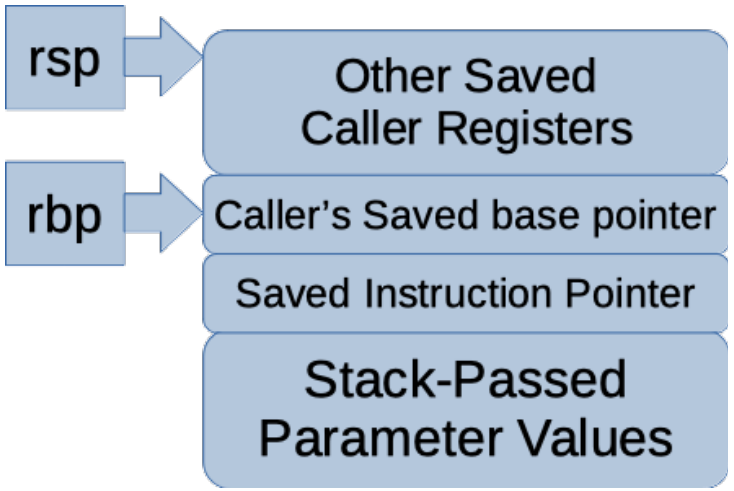
```
subroutine:
    push    %rbp
    mov     %rsp, %rbp
    push    %rbx
    push    %r13


    ...


    pop     %r13
    pop     %rbx
    pop     %rbp
    ret
```

Save some registers on the stack.

Restore them Before returning (in stack order)



Creating a Stack Frame

- We can allocate space on the stack for our own local variables.

```
void subroutine()
{
    long x = 100;
    ...
}
```

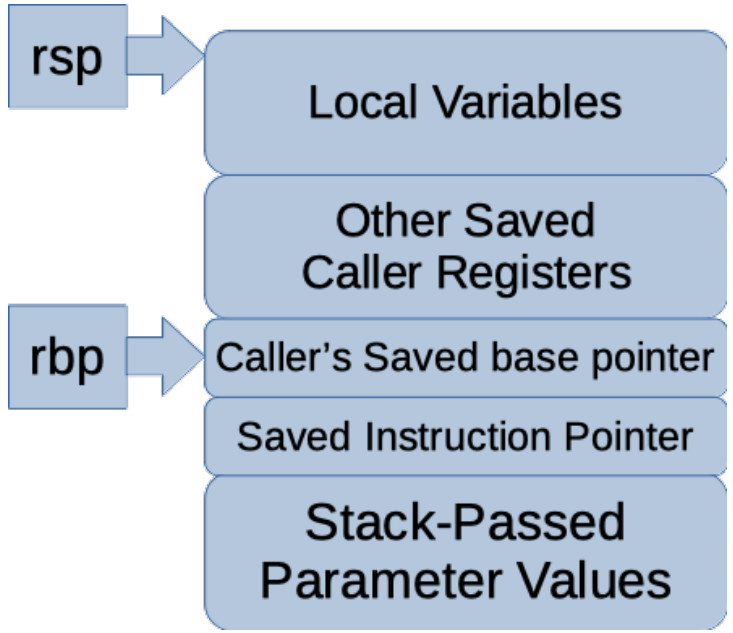
```
subroutine:
    push    %rbp
    mov     %rsp, %rbp
    push    %rbx
    push    %r13
    pushq   $100

    ...

    pop     %rax
    pop     %r13
    pop     %rbx
    pop     %rbp
    ret
```

Store a 100 on the stack.

Remove it (rax doesn't matter here)



Calling Another Function

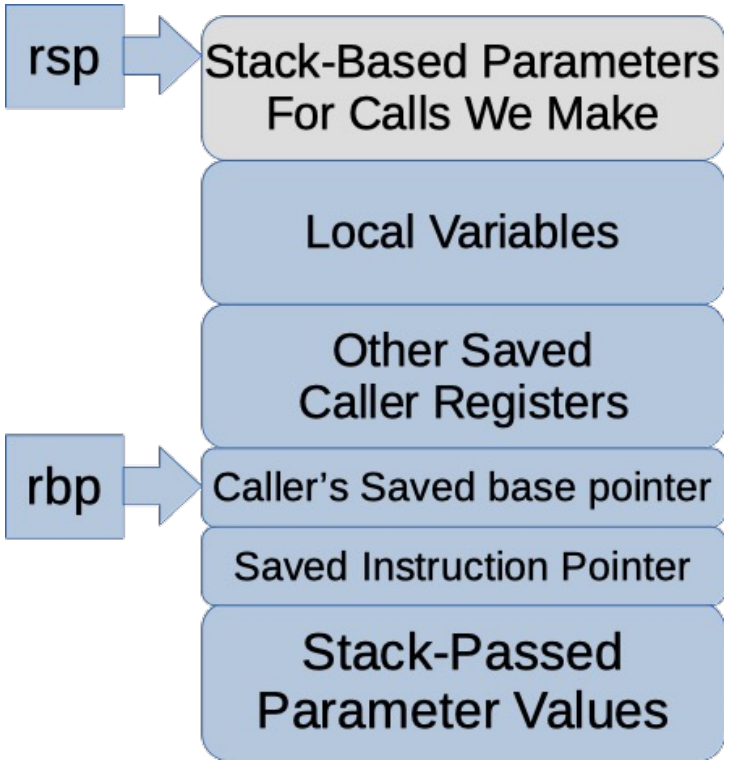
- We can make our own function calls.
- We need to stage the parameters.

```
subroutine:
    push    %rbp
    mov     %rsp, %rbp
    push    %rbx
    push    %r13
    pushq   $100
    push    aBunchOfParameters
    call    someFunciton

    pop     %rax
    pop     %r13
    pop     %rbx
    pop     %rbp
    ret
```

In general, parameters might require sack space.

Here, one register is enough.



Stack Alignment

- gcc expects the stack to be 16-byte aligned whenever you make a function call.
- The call / push %rbp maintains this.
- We need to worry about this ourselves for:
  - Saved caller registers
  - … plus local variable storage.

```
subroutine:
    push    %rbp
    mov     %rsp, %rbp

    ...

    pop     %rbp
    ret
```

Still 16-byte aligned.

